```
import pandas as pd
import nltk

nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
True
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('wordnet')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]       date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
True
```

```
import csv
csv.field_size_limit(2147483647) # Set a higher field size limit (maximum for a 32-bit integer)
df = pd.read_csv('train1.csv', engine='python')
```

```
df.iloc[0]
```

	0
id	0
title	House Dem Aide: We Didn't Even See Comey's Let...
author	Darrell Lucus
text	House Dem Aide: We Didn't Even See Comey's Let...
label	1

dtype: object

```
df.iloc[0]['text']
```

```
'House Dem Aide: We Didn’t Even See Comey’s Letter Until Jason Chaffetz Tweeted It By Darrell Lucus on October 30, 2016 Subscri
be Jason Chaffetz on the stump in American Fork, Utah ( image courtesy Michael Jolley, available under a Creative Commons-BY li
cense) \nWith apologies to Keith Olbermann, there is no doubt who the Worst Person in The World is this week–FBI Director James
Comey. But according to a House Democratic aide, it looks like we also know who the second-worst person is as well. It turns ou
t that when Comey sent his now-infamous letter announcing that the FBI was looking into emails that may be related to Hillary C
linton’s email server, the ranking Democrats on the relevant committees didn’t hear about it from Comey. They found out via a t
weet from one of the Republican committee chairmen. \nAs we now know, Comey notified the Republican chairmen and Democratic ran
```

```
text = df.iloc[0]['text']
text = text.lower()
```

```
from nltk import word_tokenize,sent_tokenize
```

```
sentences = sent_tokenize(text)
print(sentences)
```

```
['house dem aide: we didn’t even see comey’s letter until jason chaffetz tweeted it by darrell lucus on october 30, 2016 subscri
```

```
words = word_tokenize(text)
print(words)
```

```
['house', 'dem', 'aide', ':', 'we', 'didn', '’', 't', 'even', 'see', 'comey', '’', 's', 'letter', 'until', 'jason', 'chaffetz',
```

```
from nltk.corpus import stopwords
stop_words = stopwords.words('english')
```

```
print(stop_words)
```

```
['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at'
```

```
filtered = []
for word in words:
    if word.lower() not in stop_words:
        filtered.append(word)
print(filtered)
```

```
['house', 'dem', 'aide', ':', '’', 'even', 'see', 'comey', '’', 'letter', 'jason', 'chaffetz', 'tweeted', 'darrell', 'lucus', 'o
```

```
cleaned = [word for word in filtered if word.isalnum()]
print(cleaned)
```

```
['house', 'dem', 'aide', 'even', 'see', 'comey', 'letter', 'jason', 'chaffetz', 'tweeted', 'darrell', 'lucus', 'october', '30',
```

```
nltk.download('averaged_perceptron_tagger_eng')
from nltk import pos_tag
pos_tags = pos_tag(cleaned)
print(pos_tags)
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
[('house', 'NN'), ('dem', 'NN'), ('aide', 'RB'), ('even', 'RB'), ('see', 'VB'), ('comey', 'JJ'), ('letter', 'NN'), ('jason', 'NN
```

```
from nltk.stem import SnowballStemmer
```

```
stemmed = []
stemmer = SnowballStemmer('english')
for word in cleaned:
    stemmed.append(stemmer.stem(word))
print(stemmed)
```

```
['hous', 'dem', 'aid', 'even', 'see', 'comey', 'letter', 'jason', 'chaffetz', 'tweet', 'darrel', 'lucus', 'octob', '30', '2016',
```

```
from nltk.stem import WordNetLemmatizer
```

```
lm = WordNetLemmatizer()
lemmatized = []
for word in cleaned:
    lemmatized.append(lm.lemmatize(word))
print(lemmatized)
```

```
['house', 'dem', 'aide', 'even', 'see', 'comey', 'letter', 'jason', 'chaffetz', 'tweeted', 'darrell', 'lucus', 'october', '30',
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform([' '.join(lemmatized)])
print(f"Shape of TF-IDF matrix: {tfidf_matrix.shape}")
```

```
Shape of TF-IDF matrix: (1, 263)
```

```
corpus = df['text']
```

```
vectorizer = TfidfVectorizer(max_features=5000)
```

```
from collections import Counter

# Count frequency of each word
tf = Counter(cleaned)

# Normalize (important for marks)
```

```
total_words = len(cleaned)
tf_normalized = {word: count/total_words for word, count in tf.items()}

print("Term Frequency:\n", tf_normalized)
```

```
Term Frequency:
 {'house': 0.014218009478672985, 'dem': 0.004739336492890996, 'aide': 0.011848341232227487, 'even': 0.009478672985781991, 'see':
```

```
import math

documents = corpus.astype(str).tolist()   # convert to list
N = len(documents)

word_doc_count = {}

for doc in documents:
    words = set(doc.split())   # unique words
    for word in words:
        word_doc_count[word] = word_doc_count.get(word, 0) + 1

# IDF calculation
idf = {word: math.log(N / count) for word, count in word_doc_count.items()}

print("IDF:\n", dict(list(idf.items())[:20]))
```

```
IDF:
 {'out,': 2.8542994870140146, 'be': 0.294952024373235, 'agency': 2.9243064666202088, 'may': 1.2649280096252233, 'World': 2.41169
```

```
tfidf_matrix = vectorizer.fit_transform(documents)
print(tfidf_matrix.shape)
```

```
(20800, 5000)
```

```
tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=vectorizer.get_feature_names_out()
)
```

```
print("TF-IDF Matrix:\n")
print(tfidf_df)
```

```
TF-IDF Matrix:

        00       000        10  100        11        12  125        13  \
0      0.0  0.000000  0.000000  0.0  0.000000  0.000000  0.0  0.000000
1      0.0  0.000000  0.000000  0.0  0.000000  0.000000  0.0  0.000000
2      0.0  0.000000  0.000000  0.0  0.016310  0.000000  0.0  0.020081
3      0.0  0.000000  0.000000  0.0  0.000000  0.000000  0.0  0.029306
4      0.0  0.000000  0.000000  0.0  0.000000  0.000000  0.0  0.000000
...    ...       ...       ...  ...       ...       ...  ...       ...
20795  0.0  0.000000  0.000000  0.0  0.000000  0.000000  0.0  0.000000
20796  0.0  0.000000  0.000000  0.0  0.015128  0.000000  0.0  0.000000
20797  0.0  0.017190  0.017855  0.0  0.041468  0.022973  0.0  0.000000
20798  0.0  0.000000  0.036846  0.0  0.042787  0.000000  0.0  0.052681
20799  0.0  0.041251  0.000000  0.0  0.016585  0.000000  0.0  0.000000

             14        15  ...  как   мы   на   не   но   по  сша   то  что  \
0      0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
1      0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
2      0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
3      0.000000  0.099468  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
4      0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
...         ...       ...  ...  ...  ...  ...  ...  ...  ...  ...  ...  ...
20795  0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
20796  0.018691  0.015805  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
20797  0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
20798  0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0
20799  0.000000  0.000000  ...  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0

       это
0      0.0
1      0.0
2      0.0
3      0.0
4      0.0
...    ...
20795  0.0
```

```
20796  0.0
20797  0.0
20798  0.0
20799  0.0

[20800 rows x 5000 columns]
```

Start coding or generate with AI.